Layer by Layer: An Empirical Ablation of Replay Defenses in a Signed Banking API

MD. Shahriar Islam
Department of CSE
United International University
Dhaka, Bangladesh
adsh257@gmail.com

Md. Mine Uddin
Department of CSE
United International University
Dhaka, Bangladesh
md.mineuddin1212@gmail.com

Maknun Ahmed Sifat
Department of CSE
United International University
Dhaka, Bangladesh
maknunsifat2002@gmail.com

MD. Shahriar Islam, Md. Mine Uddin, and Maknun Ahmed Sifat contributed equally to this work.

***Abstract*—Replay attacks on signed transaction APIs are generally addressed using request signatures, timestamp windows, single-use nonces, and idempotency keys, but specifications do not mandate window sizes, nonce expiration times, and failure handling for requests. We analyze how these four layers combine on a repeatable banking API testbed with a ledger-based oracle for money movement. A 16-fold factorial ablation study of attack scenarios and layer configurations identifies which layer defends against which attack. The two most important findings are as follows. First, a layer of idempotency that remembers only successful requests does not protect from replaying previously failed requests because of the lack of funds; after transferring the money to the account, all 30 of such requests succeeded. Whereas the nonce check prevented fast replays and timestamp checks slow replays, the logging of every result prevents them both, albeit by disallowing retrying within the same ID. Also, nonce storage has to account for both past and future tolerance windows since the combination of default parameters (300 s nonce storage, 300 s past window, and 5 s future) allows for successful replay by the client clocked ahead of its time by 4 s within 301 s and 303 s. There is a total of about 0.5 ms per call on average for the four layers combined. All the results have been attained via single-machine simulations.**

***Index Terms*—replay attack, HMAC, nonce, idempotency, defense in depth, transaction API**

I. Introduction

Bank APIs offer the facility of transferring funds via HTTP requests. An attacker can intercept the request from a log, proxy, or TLS-termination gateway and replay the request so that the same transaction gets executed again. For example, a request for withdrawal of 1,000 BDT can be repeated. Although TLS protects information sent over the network, it does not eliminate the possibility of reusing an intercepted request. Replay attack is detrimental to the authentication and open banking protocols [1]–[4].

One of the most widely used countermeasures involves four layers. The first layer involves the use of HMAC-SHA256 signatures to help identify the changes made to the signed fields of the request. The second layer is timestamps that limit the age of the request. The third layer in this countermeasure is single-use nonce which is used to determine whether a request has been repeated or not while keeping them in record. The fourth layer is the request ID (idempotency key) layer recorded in the ledger so that a repeated id is blocked or rejected.

The RFC document indicates the key specification related to nonce, signature age, and time window used in the application [5]. The RFC 9449 is aimed at limiting the proof acceptance period and provides the replay-protection capabilities of different systems [6]. The findings obtained after the analysis of financial-grade APIs demonstrate that no information about various timing and configuration issues discussed in the current research is available [4], [7]. The researches concerned with idempotency focus on the issue of spontaneous duplicates [8], [9].

The question concerning interaction of levels in different surroundings is not considered. The errors that happen while the levels are being skipped or when the states are being retained also help solving the problem related to replaying. The way the failed requests are treated also matters: v1 API returns the state while in v2 API the state gets retried [10].

The case of S10 is an example of a replayed transaction. The same transaction is refused with respect to a certain account if it has insufficient funds in the account. In this case, the lesson is that if the state of the account is changed, then the same transaction can be executed now due to the fact that API accepts the same transaction request to be executed again, which was previously rejected. So, We address the following four research questions:

1) **RQ1 (Coverage):** What attacks can be deterred by each security layer on their own and in combination with other layers?
2) **RQ2 (Failed requests):** Is the policy for recording failed requests instrumental in determining whether the replay will succeed in case of change in the account state? Which security layers are needed to prevent the above-mentioned replays?
3) **RQ3 (Nonce retention):** What is the relationship between nonce retention and asymmetric timestamp tolerance that allows different acceptance periods of time?
4) **RQ4 (Operational effects):** What is the impact of nonce-store failure and clock skew on security and availability, and what is the cost of processing in the four security layers?

In the experiment, Anis is the honest client performing

normal withdrawals, while Eve is the attacker who captures and replays the client's transaction requests. The contributions of this study are:

1) **Coverage of security layers (RQ1):** We examine the 16 possible combinations of the four security layers with respect to 11 attacks (13 variations). The coverage matrix derived indicates the level of each layer protection from the attacks, both on its own and together with others.
2) **Replay of failed requests (RQ2):** We determine the influence of a replay of failed requests in terms of idempotency policy alternatives.
3) **Nonce retention gap (RQ3):** We determine the nonce retention gap for default settings and a significant number of combinations. While the basic idea underlies RFC 9449, our effort is in measuring the exposure in the tested system.
4) **Operational behavior and costs (RQ4):** We quantify the impact of the nonce storage failures, missing responses, and clock drifts as well as false refusals and costs of processing.

II. Related Work

Standards and Formulations of Analysis. RFC 9449 has set a limit for the time of using DPoP proofs by servers, but it allows some future timestamps to take into account clock drift, and it promotes the use of new proofs at every attempt [6]; RFC 9421 defines the parameters of *created*, *expires* and *nonce*, while maximum age, nonce unique, and time windows parameters are determined by applications [5]. Fett et al. conducted the four types of attacks against OpenID financial grade API and verified the fixed properties of the protocol [7]. Hosseyni et al. modified this work on FAPI 2.0 using DPoP and signed messages, and finding even more attacks including the DPoP proofs replay attack [4]. Both techniques leave timing parameters such as window sizes out of their models. Almehrej et al. have shown that it is possible to replay the previous replies in the Open Banking account protocol together with the usage of nonce to prevent this problem [3]. Their research is about the logic of protocols only, without reference to the variables of windows of time or length of the failures.

Replay defenses in the banking sector and their uses. The term "replay" is used by Jing et al. for the multiple instances of generating fraudulent transactions while processing 18 million bank transactions, as against the use of request replay [18]. Arta mentions the usage of HMAC-SHA256, timestamps and Redis based nonces for the REST APIs causing an increase in latency of 0.453 ms on the same host [11]. Ibrahim and Alrajh claim the capability of detecting 10,000 replayed tokens in a multi-layer architecture and with constant token validity of 120 s [12].

Freshness, clocks and idempotency. The freshness of timestamps is dependent on synchronized clocks [1]. Kerberos uses timestamps in authenticators [2]. Li et al. analyse timed protocols with parameters and identify a timing flaw in the specification of Kerberos V [14]. Krentz and Voigt analyse differing types of freshness, sequential and strong freshness in context of limited resource IoT [13]. According to Julaihi et al. finding out in IoMT simulation, the size of the window can cause legitimate acceptance at the expense of replays accepted [15]. To our knowledge, the exposure caused by a retention period shorter than the acceptance window has not been measured in a concrete implementation. RIFL records completion records permanently and restores saved results after the crash and failure of the network [8]. While Whitaker et al. use idempotency keys in service mesh system [9], the API v1 of Stripe responds and can delete the keys after 24 hours. Meanwhile, API v2 attempts recovery [10].

In the context of concurrency and failure, single use state has been studied. Nasiri et al. discover race conditions in single-use coupon use over the protocols HTTP/2 and HTTP/3, meaning that single-use must comply with the atomicity constraints [16]. Abdelhafez et al. require consistent state of single-use in a TLS 1.3 0-RTT server cluster [17]. The summary of findings is presented in Table I below, which indicates that no work covering the respective issue discusses multiple removals or ablations of signing, timestamping, nonce and idempotent layers of transaction API, nor addresses policies related to failed requests and retention.

TABLE I
RELATED WORK COMPARED WITH THIS STUDY. ✓ = STUDIED; (✓) = STUDIED PARTLY OR IN SOME OF THE WORKS; – = NOT STUDIED. AUTH: REQUEST AUTHENTICATION (SIGNATURE, MAC OR AEAD). TIME: TIMESTAMP WINDOW. NONCE: SINGLE-USE STATE. ID: IDEMPOTENCY KEY. ABL.: ABLATION OF LAYERS. FAIL: FAILED-REQUEST OR STATE-FAILURE POLICY.

Work	Domain	Method	Auth	Time	Nonce	ID	Abl.	Fail
RFC 9421, RFC 9449 [5], [6]	HTTP, OAuth	Specification	✓	✓	✓	–	–	–
Fett et al. [7]; Hosseyni et al. [4]	Financial APIs	Formal analysis	✓	–	(✓)	–	–	–
Almehrej et al. [3]	Open Banking	Model checking	–	–	✓	–	–	–
Arta [11]; Ibrahim and Alrajh [12]	REST, auth-link APIs	Implementation, measurement	✓	✓	✓	–	(✓)	–
Krentz and Voigt [13]; Li et al. [14]; Julaihi et al. [15]	IoT, protocols, IoMT	Design, formal, simulation	(✓)	✓	✓	–	–	–
Lee et al. [8]; Whitaker et al. [9]; Stripe [10]	RPC, mesh, payments	Implementation, documentation	–	–	–	✓	–	(✓)
Nasiri et al. [16]; Abdelhafez et al. [17]	Web, TLS, PKI	Measurement, implementation	–	–	(✓)	–	–	(✓)
This work	Banking API	Testbed, factorial ablation	✓	✓	✓	✓	✓	✓

III. Testbed and Methodology

A. Testbed and request pipeline

We have a small testbed with a banking API (Fig. 1), which is implemented with a Node.js/Express server providing the login, withdraw, and deposit functionalities. The accounts and

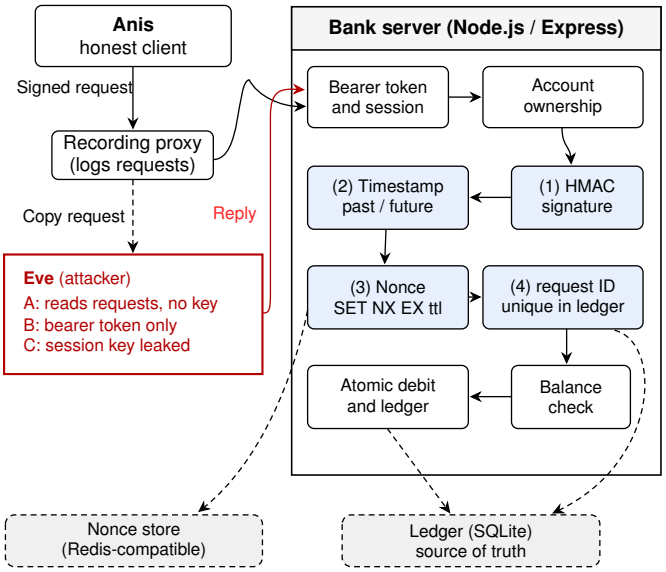


Fig. 1. Threat model and bank server request-validation workflow.

ledger information are stored in SQLite. The nonces are stored in Redis-compatible storage (Memurai 4.2.3, Windows).

During login (user ID + PIN), the server provides a bearer token and a secret session key K_s, which is not retransmitted later. All requests contain the token and: method, path, user ID, account ID, amount, currency, timestamp, nonce, request ID, and signature. Signature is calculated as HMAC-SHA256 on all other parameters using K_s. It is our own implementation, not RFC 9421 and DPoP, although based on these. Fields that contain "|" or control characters are rejected.

Once token ownership validation is completed, there are four additional stages of protocol flow. These stages are denoted H, T, N, and R respectively. **H** (signature) is stage during which HMAC hash is recomputed and compared. **T** (timestamp) is stage of timestamp verification; which allows maximum PAST clock time in seconds from the transaction which occurred earlier and maximum FUTURE clock time in seconds until the transaction takes place (default values are 300 sec and 5 sec respectively). **N** (nonce) is nonce verification stage; nonce is stored with TTL or time to live limit of a certain duration (default is 300 sec). Finally **R** (request ID) is ID verification which looks up the ID stored in the ledger and any duplicates are rejected. H1T0N1R0 means that only H and N are on, all other stages are off.

B. Threat model and attack scenarios

The adversary is called Eve. The attacker A can alter any field in a request including token (from proxy logs or through intermediaries like proxies/gateways stripping out TLS the protocol for Web encryption) but does not know the session key and thus cannot generate a valid signature. Within the test-bed environment, Eve replays only those request that are recorded within the localhost proxy. The attacker B knows only bearer token while the attacker C knows both the session key as well. Out of scope for this research would be compromised clients, denial-of-service attacks, PIN guessing, and multi server architecture. A successful attack results when there is an additional SUCCESS withdrawal transaction on the ledger. Table II lists the scenarios.

TABLE II
ATTACK SCENARIOS AND NUMBER n OF TRIALS PER CONFIGURATION (S10: 30 INDEPENDENT FAILED REQUESTS).

ID	Eve's action	n
S1	Resend a captured request immediately	30
S2	Resend after 2 s (inside the window)	30
S3	Clear the nonce store, then resend	30
S4	Resend after 7 s (outside the 6 s window)	30
S5	Send 50 identical copies in parallel, 30 batches	1,500
S6	Change the amount, keep the old signature	30
S7a, S7b	Change request ID (a) or nonce (b), keep the old signature	30 each
S8	Resend after the session expired	30
S9	Attacker C: new request signed with the leaked key	30
S10	Keep failed (insufficient balance) requests; after a deposit, replay each once, 1 s (fast) or 9 s (late) later	30
S11	Attacker B: token only, fresh nonce, request ID and timestamp, no signature	100

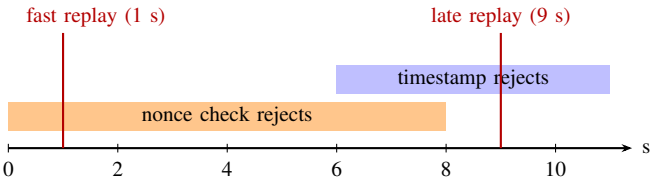


Fig. 2. S10 timeline (scaled settings, schematic). Eve copies a failed request, a deposit follows, then Eve replays it. Orange: the nonce is still remembered (8 s). Blue: the request is older than the 6 s past limit. A fast replay is stopped only by the nonce check, a late replay only by the timestamp check. Delays are counted from the deposit; the failed requests were sent just before it.

C. Experiments

Coverage experiment (E1). We test each of the 16 combinations of on/off for the 4 layers in all scenarios in Table II, using different servers and databases. Since it is impractical to wait 300 seconds for each experiment, the coverage experiment and retention experiment used scaled parameters: PAST 6 seconds, FUTURE 2 seconds, nonce TTL 8 seconds, session duration 15 seconds. In other runs of the same experiment, normal parameters were employed. The result was consistent for all trials, and we report results with 95% Wilson confidence intervals (0 out of 30 corresponds to true rate not greater than 11.4%; 0 out of 100 means not more than 3.7%).

Failed request experiment (S10). There are 30 failed attempts to withdraw 100 amount of money every time with an INSUFFICIENT_BALANCE error. The balance at this time is 1. In the next phase, a deposit of 3001 is made to the account. Eve replays every failed request after 1 second (fast) or 9 seconds (slow) after making a deposit, according to the two policies. In `any_outcome` case, retries are made in two possible ways- either with same ID and new nonce or with different and new ID (Fig. 2).

Retention test (E4a). With H, T and N functioning, we change the nonce TTL (4, 6, 8, 10, 12 s), the time difference of the client clock f (0, 1, 2 s) and the delay time d (1, 3, 5, 6, 7, 8, 9, 10, 11 s), conducting 30 trials for each delay. After

reaching the point when $d \geq \mathrm{TTL}$, the nonce no longer exists in memory. Nevertheless, until $d < \mathrm{PAST} + f$, the request still seems to be new, as the clock ahead by f seconds makes the request appear new. So a replay succeeds exactly when

$$\mathrm{TTL} \leq d < \mathrm{PAST} + f, \tag{1}$$

When the nonce TTL is at least PAST + FUTURE, the gap in Eq. (1) closes. For the full-scale check, we used the default settings (TTL 300 s, PAST 300 s, FUTURE 5 s), with the client clock 4 s ahead and replay delays of 299, 301, 303, 305 and 307 s, with R on and off.

Cost testing (E2). in eight variations: none, with H, with H and T, with H and T and N, and so forth (in total, all possible combinations), with the total of 10 repetitions per each case (in which the time is recorded in the server).

Test of clock-error (E3). It simulates genuine requests. The time-stamped sending time (i.e., when the request message is sent) is computed by adding the offset of the client clock. The server gets the message after some delay. The stage of the request at the server is given by the following formula: delay$-$offset. The server accepts this request when $-\mathrm{FUTURE} \leq$ age $\leq$ PAST. Types of requests with time periods of PAST $\in \{5, 30, 60, 120, 300\}$ s, and FUTURE $\in \{0, 2, 5, 30, 60\}$ s are included in combinations with three alternative profiles containing a total of 75 cells and the number of requests. The delays are log-normally distributed according to the certain profile. The offset is normal, $N(0, \sigma)$; the delay is log-normal with a median and a shape. P1: $\sigma = 0.5$ s, median 0.2 s, shape 0.6. P2: 3 s, 0.5 s, 1.0. P3: 30 s, 2 s, 1.2. Each cell draws its own requests, so differences below about one percentage point are noise.

Testing Failure (E4b, E4c). The nonce store is disrupted through a route that tests only cases with 30 honest requests and 30 replays each. When a response goes missing from the server, the transaction will process but the client will not receive the information and abort the attempt. Each cell will consist of 200 transactions during which responses are discarded with proportions $p = 0$, 0.05 and 0.2, in the order given or by the request ID provided, while the nonce is the same or different in retries.

Additional Tests. We also conducted a witness group of honest requests and checked for route binding and atomic debit as well as reporting any time-based issues present in the testing phase. To this end, the testing harness contains 44 tests and a separate analysis script whereby 4,532 values were computed with no differences observed. The code and data can be found here: `github.com/shahriarshad/Replay-Attack-Simulation`.

IV. Results

A. Coverage of the four layers (E1)

Question: which attacks does each layer block? **Answer:** no single layer can block all attacks; however, when the four layers are used together, all attacks can be blocked except the one involving key leakage (S9) (see Fig. 3).H does not block replay attacks (S1 through S5 are not blocked by H alone

		S1,S2 n=30	S3 n=30	S4 n=30	S5 n=1,500	S6 n=30	S7a n=30	S7b n=30	S10 fast n=30	S10 late n=30	S11 n=100
0 on	H0T0N0R0	30	30	30	1,500	30	30	30	30	30	100
1 on	H0T0N0R1	0	0	0	0	0	30	0	30	30	100
	H0T0N1R0	0	30	0	0	0	0	30	0	30	100
	H0T1N0R0	30	30	0	1,500	30	30	30	30	0	100
	H1T0N0R0	30	30	30	1,500	0	0	0	30	30	0
2 on	H0T0N1R1	0	0	0	0	0	0	0	0	30	100
	H0T1N0R1	0	0	0	0	0	30	0	30	0	100
	H0T1N1R0	0	30	0	0	0	0	30	0	0	100
	H1T0N0R1	0	0	0	0	0	0	0	30	30	0
	H1T0N1R0	0	30	0	0	0	0	0	0	30	0
	H1T1N0R0	30	30	0	1,500	0	0	0	30	0	0
3 on	H0T1N1R1	0	0	0	0	0	0	0	0	0	100
	H1T0N1R1	0	0	0	0	0	0	0	0	30	0
	H1T1N0R1	0	0	0	0	0	0	0	30	0	0
	H1T1N1R0	0	30	0	0	0	0	0	0	0	0
4 on	H1T1N1R1	0	0	0	0	0	0	0	0	0	0

Fig. 3. Coverage matrix (E1): successful attacks out of n trials for all 16 configurations (H = HMAC, T = timestamp, N = nonce, R = request ID; 1 = on). Red means at least one attack succeeded; the number is the count. Scaled profile (PAST 6 s, FUTURE 2 s, nonce TTL 8 s), policy `success_only`. S8 (0/30) and S9 (30/30) are identical in all rows and omitted.

(H1T0N0R0)), but H is effective in blocking S11. Both S6 and S7a and S7b use other identifiers, meaning that N or R can also stop them. N is effective in stopping S1, S2, and S5 and, given that TTL can go back to PAST, S4 too. S3 cannot be stopped though, and it is bypassed by the new nonce once H is turned off (S7b). T only stops S4 and the late S10. R stops S1 through S6, but S7a cannot be stopped without H, and `success_only` means S10 is not stopped. H plus R stop everything except S9 and S10. S8 was 0/30 and S9 was 30/30 irrespective of the configuration. Honest control requests were accepted 30/30 in all configurations.

B. The failed-request replay and idempotency policy (S10)

Question: The question is whether the failed-request policy determines if a replay is valid after the account has been credited. **Answer:** The answer is in(Table III). `success_only` provides no protection against S10: with N and T disabled, all 30 failed requests were executed after the deposit, irrespective of how fast or slow these requests were (Wilson interval [0.886, 1.000]). N alone is effective against fast replays (the nonce has expired), T alone against slow ones, while both N and T together are effective against either type. Under `any_outcome`, the protection of R is absolute (0/30). We also ran S10 for all 16 layer configurations under both policies; under `any_outcome`, every configuration with R on blocked both fast and late replays (0/30).

C. Nonce retention and timestamp window (E4a)

Question: What is the required retention time of a nonce? **Answer:** The value for nonce retention must be at a minimum of PAST + FUTURE. All 30, 60, 90 successful replays out of 270 were recorded in case the R is off at TTL 4 s (f = 0, 1, 2 s) while all 0, 30, 60 successful replays were recorded at TTL 6 s and 0 for TTL 8, 10 and 12 s. Meanwhile, with R on, all replays were recorded as 0. Rule (1) fulfilled the requirement of 135 on 135 cases. Also, in the full run with R off (Fig. 4(b)), the successful runs of 299 s were stopped

TABLE III

S10: EXECUTED REPLAYS OF 30 FAILED REQUESTS BY POLICY AND LAYERS (TOP; H AND R ANY), AND RETRIES WITH A NEW NONCE AFTER THE DEPOSIT, ACCEPTED OF 30 (BOTTOM).

Policy	Layers on	Fast	Late
`success_only`	N, T off	30/30	30/30
`success_only`	T only	30/30	0/30
`success_only`	N only	0/30	30/30
`success_only`	N and T	0/30	0/30
`any_outcome`	R on	0/30	0/30
		Same ID	New ID
`success_only`		30/30	30/30
`any_outcome`		0/30	30/30

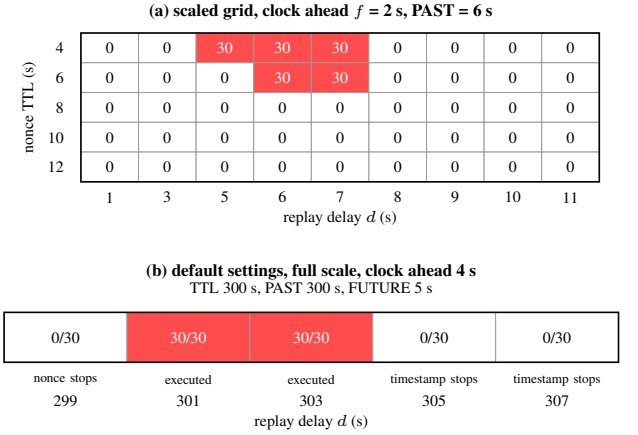


Fig. 4. Nonce retention gap (E4a): replays executed out of 30 per cell (red), R off. (a) Scaled grid, client clock 2 s ahead: replays succeed for TTL $\leq d <$ PAST $+f$. (b) Default settings at full scale, clock 4 s ahead: replays succeed between 300 s and 304 s.

because of N, 301 s and 303 s were successful (30/30), while 305 s and 307s were stopped by T. With R being activated, all five delays resulted in a 0/30 (ALREADY_PROCESSED at 301 seconds and 303 seconds). The default TTL (at 300 seconds) corresponds to PAST and misses PAST+FUTURE by 5 seconds and the server displays a startup warning. With all four layers turned on at default parameters, replays at the 60-second mark (S2) and the 301-second mark (S4) recorded a 0/30 status.

D. Layers Cost (E2)

Question: what are the costs of the layers? **Answer:** In essence, the cost of the layers is around 0.5 ms per request, split mainly between the request-ID lookup (240 μs) and the nonce store (190 μs) (Table IV).The four layers' own time sums to 0.48 ms and the end-to-end handler time rises by 0.62 ms (6.07 to 6.69 ms, around 10%; a repeat run gave 6.67 ms), because the second request-ID check is not included in the per-layer sum. Rejected requests are much cheaper than accepted ones: wrong signature 0.73 ms, stale timestamp 0.72 to 0.78 ms, duplicate nonce 0.94 to 1.27 ms, duplicate request ID 1.06 to 1.28 ms, versus 6 to 7 ms for an accepted request.The numbers presented are for comparative purposes only.

TABLE IV

COST (E2): SERVER HANDLER TIME (MS, MEDIAN OF REPEAT MEDIANS) AND PER-LAYER TIME (μS, ALL FOUR LAYERS ON).

Configuration	ms	Layer (own time)	μs
None	6.07	HMAC	37
H	6.13	Timestamp	10
H+T	6.13	Nonce (Redis-compat.)	190
H+T+N	6.32	request ID	240
All four	6.69	Sum	477
All four, memory nonce	6.45	Nonce, memory	16
All four, SQLite nonce	11.22	Nonce, SQLite	4,517

TABLE V

FAILURE BEHAVIOUR: NONCE-STORE FAILURE (E4B, 30 LEGITIMATE REQUESTS AND 30 REPLAYS) AND LOST RESPONSES (E4C, 10 AND 40 LOST).

Mode	R	Legit. accepted	Replay succeeded
Fail-open	off	30/30	30/30
Fail-open	on	30/30	0/30
Fail-closed	off or on	0/30	0/30
Lost response, retry		Error	Success inferable
Default order, identical		NONCE_FAIL	0 of 10 / 0 of 40
R first, or new nonce		ALREADY_PROC.	10 of 10 / 40 of 40

E. False rejection of honest customers (E3)

Question: what is the number of honest clients that are wrongly rejected? **Answer:** around one third at FUTURE = 0; small FUTURE corrects it, but small PAST rejects clients due to their slow clocks. At FUTURE = 0 the rejection rate varied between 31% - 33%, although the clients were well synchronized using profile P1 since being a little too fast is interpreted as future; however, by setting FUTURE to 2 seconds or longer this particular issue was resolved. In the case of severe profile P3, with PAST = 300 s and FUTURE = 5 s, the rejection was 38.9%, while with FUTURE set to 60 s the rejection dropped to 0.95% (19 of 2,000 requests, P3, PAST = 300 s).

F. Malfunction behavior (E4b, E4c)

Question: what do we do when the nonce store fails or the response is lost? **Answer:** the fail-open mode lets service continue to operate but lets replays through unless R is turned on. When responses are lost nothing is charged twice but the error may not be obvious (as can be seen from (Table V)). If the store fails while R is off, the fail-open mode lets all 30 requests go through successfully while fail-closed rejects all legitimate requests and all replays. When responses are lost, each request ID is charged only once (200 requests from 200) for 0, 10 and 40 lost responses.With the default ordering and a perfect retry the error message was NONCE FAIL so the client could not tell that the prior request had succeeded (0 of 10, 0 of 40). With R first or a new-nonce retry the error message was ALREADY_PROCESSED allowing the client to know that the earlier request had been successful (success inferable in 10 of 10 and 40 of 40 retries).

G. Correctness and resource checks

In the correctness and resource validation tests, a signed deposit sent to the withdrawal endpoint, and vice versa, was denied with HTTP status 400, rendering no ledger rows (in all 4 test cases, whether Testing was ON or OFF). With a balance of 100, 100 parallel withdrawals of 1 each were repeated 30 times without the balance going negative. There were a total of 1,000 iterations of 50 identical requests, where only one request was accepted from each batch. A request with age 299 s was accepted while a request of age 301 s was refused. A request with a timestamp of 5 s ahead was accepted whereas the request with age of 6 s ahead was rejected.

V. Discussion and Limitations

Lessons. (i) Preserve nonce retention = PAST + FUTURE. PAST alone isn't enough with a future tolerance. (ii) Maintain a small but non zero FUTURE tolerance, because FUTURE = 0 fails 1/3 of honest clients with P1. (iii) Intentionally decide on an idempotency policy and also record failures (returning the same result would feel natural but has never been tested), or preserve checks both on nonce and time. (iv) Never use nonce without a signature (S7b). (v) Pick fail-open or fail-closed nonce storage and use a durable request ID layer with fail-open. (vi) Early check the request ID for unambiguous errors in case of response loss. Make sessions short-lived and keep keys secret. S9 proves that none of the layers is useful when the key leaks. The assertion of necessity of all four layers applies only to `success_only`. By combining E1 (works with `success_only`) and S10, our results suggest that `any_outcome` with HMAC and request ID defeats all attacks except S9.

Limitations. This is a local Windows laptop emulation using Memurai (instead of Redis) for comparison. Samples are small ($0/30 \leq 11.4\%$, $0/100 \leq 3.7\%$), although results were identical among all the experiments. E1 (S1-S9, S11) was run under `success_only`; S10 was run for all 16 configurations under both policies. E1 and grid experiments use scaled time. Default-window experiments involve S2, S4, and retention window but not S10 late. Distribution of E3 is assumed. One process with transaction lock provides race results. Multi-process and distributed setups are not included. The E4b table uses 30 replays per cell. Earlier runs with one replay per cell were superseded. Tests were done using HTTP with one user, light load, no mixed traffic, and faults injected within the process.

VI. Conclusion

Every replay defense layer covers up a different gap, and leftover gaps are due to configuration decisions. In our setup, an idempotency layer that disregards failed requests created replayable failures unless both nonce and timestamp tests were enabled. Equal nonce lifetime to past window resulted in a replay gap of the size of the future window tolerance, at default settings. The timers of all four layers together amount to about 0.5 ms per request. The results were obtained using simulation on one machine; distributed experiments are future work.

References

[1] L. Gong, "A security risk of depending on synchronized clocks," *ACM SIGOPS Oper. Syst. Rev.*, vol. 26, no. 1, 1992, doi: 10.1145/130704.130709.

[2] C. Neuman, T. Yu, S. Hartman, and K. Raeburn, "The Kerberos network authentication service (V5)," IETF, RFC 4120, 2005, doi: 10.17487/RFC4120.

[3] A. Almehrej, L. Freitas, and P. Modesti, "Account and transaction protocol of the Open Banking standard," in *Rigorous State-Based Methods (ABZ 2020)*, Lecture Notes in Computer Science, vol. 12071. Springer, 2020, pp. 230–236, doi: 10.1007/978-3-030-48077-6_16.

[4] P. Hosseyni, R. Küsters, and T. Würtele, "Formal security analysis of the OpenID FAPI 2.0 family of protocols: Accompanying a standardization process," *ACM Trans. Priv. Secur.*, vol. 28, no. 1, art. 4, 2024, doi: 10.1145/3699716.

[5] A. Backman, J. Richer, and M. Sporny, "HTTP message signatures," IETF, RFC 9421, Feb. 2024, doi: 10.17487/RFC9421.

[6] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones, and D. Waite, "OAuth 2.0 demonstrating proof of possession (DPoP)," IETF, RFC 9449, Sep. 2023, doi: 10.17487/RFC9449.

[7] D. Fett, P. Hosseyni, and R. Küsters, "An extensive formal security analysis of the OpenID financial-grade API," in *Proc. IEEE Symp. Security and Privacy*, 2019, doi: 10.1109/SP.2019.00067.

[8] C. Lee, S. J. Park, A. Kejriwal, S. Matsushita, and J. Ousterhout, "Implementing linearizability at large scale and low latency," in *Proc. 25th ACM Symp. Operating Systems Principles (SOSP)*, 2015, doi: 10.1145/2815400.2815416.

[9] M. Whitaker, B. Volckaert, and M. Al-Naday, "Idempotency in service mesh: For resiliency of fog-native applications in multi-domain edge-to-cloud ecosystems," in *Proc. 15th Int. Conf. Cloud Computing and Services Science (CLOSER)*, SCITEPRESS, 2025, pp. 182–189, doi: 10.5220/0013293900003950.

[10] Stripe, Inc., "Idempotent requests," Stripe API Reference. [Online]. Available: https://docs.stripe.com/api/idempotent_requests (accessed Oct. 2026).

[11] M. D. Arta, "Implementasi mekanisme autentikasi API berbasis HMAC-SHA256 dengan skema nonce dan timestamp untuk mencegah replay attack," Institut Teknologi Bandung, 2025, Institut Teknologi Bandung.

[12] N. Ibrahim and A. Alrajh, "A multi-layered security framework for protecting authentication link generation APIs through integrated defence in depth mechanisms," *Int. J. Intell. Eng. Syst.*, vol. 19, no. 5, pp. 985–1004, 2026, doi: 10.22266/ijies2026.0531.56.

[13] K.-F. Krentz and T. Voigt, "More lightweight, yet stronger: Revisiting OSCORE's replay protection," in *Proc. Workshop on Security and Privacy in Standardized IoT (SDIoTSec)*, NDSS Symposium, 2024, doi: 10.14722/sdiotsec.2024.23003.

[14] L. Li, J. Sun, Y. Liu, M. Sun, and J. S. Dong, "A formal specification and verification framework for timed security protocols," *IEEE Trans. Softw. Eng.*, vol. 44, no. 8, pp. 725–746, 2018, doi: 10.1109/TSE.2017.2712621.

[15] A. A. Julaihi *et al.*, "SELAM: Selective ECC-based lightweight authentication for the internet of medical things," *Sci. Rep.*, vol. 16, art. 20531, 2026, doi: 10.1038/s41598-026-53641-x.

[16] M. A. Nasiri, E. Chatzoglou, and G. Kambourakis, "QUIC-er races: HTTP/3 won't save you from TOCTOU vulnerabilities," *Int. J. Inf. Secur.*, vol. 25, art. 83, 2026, doi: 10.1007/s10207-026-01258-6.

[17] M. E. Abdelhafez, S. Ramadass, and M. Abdelwahab, "TLS Guard for TLS 1.3 zero round-trip time (0-RTT) in a distributed environment," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 35, art. 101797, 2023, doi: 10.1016/j.jksuci.2023.101797.

[18] C. Jing, C. Wang, and C. Yan, "Replay attack: A prevalent pattern of fraudulent online transactions," in *Proc. 5th IEEE Int. Conf. Cyber Security and Cloud Computing*, 2018, pp. 75–82, doi: 10.1109/CSCloud/EdgeCom.2018.00022.